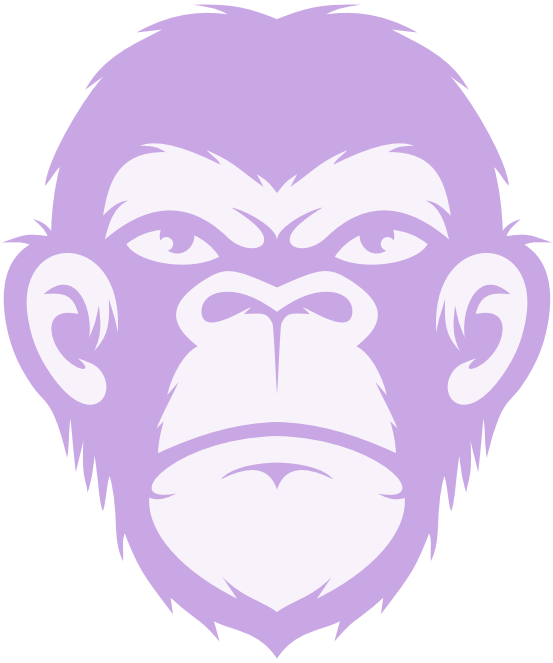

TSwap Audit Report

Version 1.0

Sebi de la Mata

May 12, 2026

TSwap Audit Report

Sebi de la Mata

May 12th, 2026

Prepared by: Sebi de la Mata Lead Auditors: - Sebi de la Mata

Table of Contents

Protocol Summary

TSwap is a Uniswap V1 style DEX. Protocol invariant is `x * y = k`.

Disclaimer

The Sebi de la Mata team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

##The Findings in this document correspond with the following commit hash:##

```
f426f57731208727addc20adb72cb7f5bf29dc03
```

Scope

```
./src/
#-- src/PoolFactory.sol
#-- src/TSwapPool.sol
```

Roles

```
- Liquidity Provider: Supplies liquidity to a token pool in order to
    collect pool fees.
- Users: Traders looking to swap one token for another.
```

Executive Summary

We spent 2 hours with one auditor using Foundry to review the scoped code. Four High level findings, one Medium finding, two Low findings, and three Informational findings were observed. One finding may require a rethink of the protocol design due to its impact.

Issues found

Severity	Number of issues found
High	4
Medium	1
Low	2

Severity	Number of issues found
Informational	3
Total	10

Findings

High

[H-1] Incorrect fee calculation in `TSwapPool::getInputAmountBasedOnOutput` protocol to take too many tokens from users, resulting in lost fees.

Description: The `getInputAmountBasedOnOutput` is intended to give the user the amount of input tokens needed to receive a desired amount of an output token. However, the function currently miscalculates the resulting amount. When calculating the fee it scales the fee by `10_000` instead of `1_000`.

Impact: Protocol takes more fees than expected from users.

Recommended Mitigation:

Scale value by `1_000` instead of `10_000`:

```
function getInputAmountBasedOnOutput(
        uint256 outputAmount,
        uint256 inputReserves,
        uint256 outputReserves
    )
        public
        pure
        revertIfZero(outputAmount)
        revertIfZero(outputReserves)
        returns (uint256 inputAmount)
    {
        return
-           ((inputReserves * outputAmount) * 10000) /
+.          ((inputReserves * outputAmount) * 1_000) /
            ((outputReserves - outputAmount) * 997);
    }
```

[H-2] Lack of slippage protection in `TSwapPool::swapExactOutput` causes users to potentially receive a much lower amount of tokens than expected.

Description: The `swapExactOutput` function does not include any sort of slippage protection. This function is similar to what is done `TSwapPool::swapExactInput`, where the function specifies a `minOutputAmount`, the `swapExactOutput` function should specify a `maxInputAmount`.

Impact: If the market conditions change before the transaction processes, the user may receive much less tokens than expected.

Proof of Concept: 1. The price of WETH is now 1,000 USDC. 2. User calls `swapExactOutput` looking for 1 WETH 3. Third-party transaction shifts pool to where 1 WETH is worth 10,000 USDC 4. User receives 1 WETH for 10,000 while expecting to only pay 1,000 USDC.

Recommended Mitigation:

```
function swapExactOutput(
    IERC20 inputToken,
+   uint256 maxInputAmount
    IERC20 outputToken,
    uint256 outputAmount,
    uint64 deadline
)
    public
    revertIfZero(outputAmount)
    revertIfDeadlinePassed(deadline)
    returns (uint256 inputAmount)
{
    uint256 inputReserves = inputToken.balanceOf(address(this));
    uint256 outputReserves = outputToken.balanceOf(address(this));

    inputAmount = getInputAmountBasedOnOutput(
        outputAmount,
        inputReserves,
        outputReserves
    );
+   if(maxInputAmount < inputAmount){
+       revert();
+   }

    _swap(inputToken, inputAmount, outputToken, outputAmount);
}
```

[H-3] `TSwap::sellPoolTokens` mismatches input and output tokens, causing users to receive the incorrect amount of tokens.

Description: The `sellPoolTokens` function is intended to allow users to sell pool tokens in exchange for WETH. The function mistakenly calls the `swapExactOutput` function when it should call the `swapExactInput` function.

Impact: Users will swap the wrong amount of tokens, causing severe disruption to protocol operations.

Recommended Mitigation:

```
function sellPoolTokens(
-   uint256 poolTokenAmount
+   uint256 poolTokenAmount,
+   uint256 minWethToReceive
) external returns (uint256 wethAmount) {
    return
-       swapExactOutput(
+       swapExactInput(
+           i_poolToken,
+           poolTokenAmount,
+           i_wethToken,
+           minWethToReceive,
+           uint64(block.timestamp)
+       );
}
```

[H-4] In `TSwapPool::swap` the extra token given to users after every `swapCount` breaks the protocol invariant of `x * y = k`

Description: The protocol follows a strict invariant of `x * y = k`, where:

- `x`: the balance of the pool token
- `y`: the balance of WETH
- `k`: the constant product

This means that whenever balances change in the protocol, the product of those balance should still equal `k` (not including fees). However, the `swap` function awards an extra token to the user every 10 swaps, meaning over time the pool will be drained of funds.

Impact: A user could maliciously drain the pool of funds by calling the `swap` function repeatedly to receive the incentive until the pool is drained.

Proof of Concept:

Paste the following code into the `TSwapPool.t.sol` unit test suite:

```
function testInvariantBroken() public {
    vm.startPrank(liquidityProvider);
    weth.approve(address(pool), 100e18);
    poolToken.approve(address(pool), 100e18);
    pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
    vm.stopPrank();

    uint256 outputWeth = 1e17;
    int256 startingY = int256(weth.balanceOf(address(pool)));
    int256 expectedDeltaY = int256(-1) * int256(outputWeth);

    vm.startPrank(user);
    poolToken.approve(address(pool), type(uint256).max);
    pool.swapExactOutput(
        poolToken,
        weth,
        outputWeth,
        uint64(block.timestamp)
    );
    pool.swapExactOutput(
        poolToken,
        weth,
        outputWeth,
        uint64(block.timestamp)
    );
    pool.swapExactOutput(
        poolToken,
        weth,
        outputWeth,
        uint64(block.timestamp)
    );
    pool.swapExactOutput(
        poolToken,
        weth,
        outputWeth,
        uint64(block.timestamp)
    );
    pool.swapExactOutput(
        poolToken,
        weth,
        outputWeth,
        uint64(block.timestamp)
    );
    pool.swapExactOutput(
        poolToken,
        weth,
        outputWeth,
        uint64(block.timestamp)
    );
```

```
50      pool.swapExactOutput(
51          poolToken,
52          weth,
53          outputWeth,
54          uint64(block.timestamp)
55      );
56      pool.swapExactOutput(
57          poolToken,
58          weth,
59          outputWeth,
60          uint64(block.timestamp)
61      );
62      pool.swapExactOutput(
63          poolToken,
64          weth,
65          outputWeth,
66          uint64(block.timestamp)
67      );
68      vm.stopPrank();
69
70      uint256 endingY = weth.balanceOf(address(pool));
71      int256 actualDeltaY = int256(endingY) - int256(startingY);
72      assertEq(actualDeltaY, expectedDeltaY);
73  }
```

Run the test in the terminal:

```
1  forge test --mt testInvariantBroken -vvvv
```

The test will fail.

This is the line causing the invariant to fail:

```
1  swap_count++;
2  if (swap_count >= SWAP_COUNT_MAX) {
3      swap_count = 0;
4      outputToken.safeTransfer(msg.sender, 1_000_000_000_000_000_000);
5  }
```

Recommended Mitigation:

Either modify the product constant invariant or remove the logic for the incentive from the swap:

```
1  - swap_count++;
2  - if (swap_count >= SWAP_COUNT_MAX) {
3  -     swap_count = 0;
4  -     outputToken.safeTransfer(msg.sender, 1_000_000_000_000_000_000);
5  - }
```

Medium

[M-1] `TSwapPool::deposit` is missing deadline check, causing transactions to complete even after the deadline

Description: The `deposit` function accepts a `uint64 deadline` parameter that according to documentation:

```
1 /// @param deadline The deadline for the transaction to be completed by
```

However, the parameter is unused within the function, allowing the transaction to be executed after the deadline has passed.

Impact: Transactions could execute when market conditions are unfavorable to the depositor, even after provide a deadline to prevent such unexpected deposits.

Proof of Concept: The `uint64 deadline` parameter is unused.

Recommended Mitigation: Add a require statementto exit the function if the deadline has passed.

```
1   /// @param deadline The deadline for the transaction to be completed
        by
2       function deposit(
3           uint256 wethToDeposit,
4           uint256 minimumLiquidityTokensToMint,
5           uint256 maximumPoolTokensToDeposit,
6           uint64 deadline
7       )
8           external
9           revertIfZero(wethToDeposit)
10          returns (uint256 liquidityTokensToMint)
11      {
12  +       if(block.timestamp > uint256(deadline)){
13  +           revert();
14  +       }
15          if (wethToDeposit < MINIMUM_WETH_LIQUIDITY) {
16              revert TSwapPool__WethDepositAmountTooLow(
17                  MINIMUM_WETH_LIQUIDITY,
18                  wethToDeposit
19              );
20          }
21      }
```

Low

[L-1] `TSwapPool::LiquidityAdded` had parameters out of order, causing events to log incorrect information.

Description: When the `LiquidityAdded` event is emitted in the `TSwapPool::_addLiquidityMintAndTrans` function the parameters are entered in the wrong order. The `poolTokensToDeposit` value should go in the third parameter, and the `wethToDeposit` value should be entered as the second parameter.

Impact: Event emmission is incorrect, postentially leading to misinterpretation of log data at a later point.

Proof of Concept:

```
1     /// @dev This is a sensitive function, and should only be called by
          addLiquidity
2     /// @param wethToDeposit The amount of WETH the user is going to
         deposit
3     /// @param poolTokensToDeposit The amount of pool tokens the user
         is going to deposit
4     /// @param liquidityTokensToMint The amount of liquidity tokens the
          user is going to mint
5     function _addLiquidityMintAndTransfer(
6         uint256 wethToDeposit,
7         uint256 poolTokensToDeposit,
8         uint256 liquidityTokensToMint
9     ) private {
10        _mint(msg.sender, liquidityTokensToMint);
11        emit LiquidityAdded(msg.sender, poolTokensToDeposit,
             wethToDeposit);
12    }
```

Recommended Mitigation: Correct the order of the input parameters:

```
1     /// @dev This is a sensitive function, and should only be called by
          addLiquidity
2     /// @param wethToDeposit The amount of WETH the user is going to
         deposit
3     /// @param poolTokensToDeposit The amount of pool tokens the user
         is going to deposit
4     /// @param liquidityTokensToMint The amount of liquidity tokens the
          user is going to mint
5     function _addLiquidityMintAndTransfer(
6         uint256 wethToDeposit,
7         uint256 poolTokensToDeposit,
8         uint256 liquidityTokensToMint
9     ) private {
10        _mint(msg.sender, liquidityTokensToMint);
```

```
11 -       emit LiquidityAdded(msg.sender, poolTokensToDeposit,
       wethToDeposit);
12 +.      emit LiquidityAdded(msg.sender, wethToDeposit,
       poolTokensToDeposit);
13         }
```

[L-2] Value returned by `TSwapPool::swapExactInput` results in incorrect return value given.

Description: The `swapExactInput` function is expected to return the amount of output tokens received by the caller. However, while it declares the named return value `output`, it is never assigned a value, nor uses an explicit return statement.

Impact: The return value of `swapExactInput` will always be `0`, giving incorrect information to the caller.

Recommended Mitigation:

```
1  function swapExactInput(
2          IERC20 inputToken,
3          uint256 inputAmount,
4          IERC20 outputToken,
5          uint256 minOutputAmount,
6          uint64 deadline
7      )
8          public
9          revertIfZero(inputAmount)
10         revertIfDeadlinePassed(deadline)
11 -       returns (uint256 output)
12 +       returns (uint256)
13     {
14         uint256 inputReserves = inputToken.balanceOf(address(this));
15         uint256 outputReserves = outputToken.balanceOf(address(this));
16
17         uint256 outputAmount = getOutputAmountBasedOnInput(
18             inputAmount,
19             inputReserves,
20             outputReserves
21         );
22
23         if (outputAmount < minOutputAmount) {
24             revert TSwapPool__OutputTooLow(outputAmount,
                   minOutputAmount);
25         }
26
27         _swap(inputToken, inputAmount, outputToken, outputAmount);
28 +.      return(outputAmount);
29     }
```

Informationals

[I-1] Unused Custom Error `PoolFactory::PoolFactory__PoolDoesNotExist`

```
- error PoolFactory__PoolDoesNotExist(address tokenAddress);
```

[I-2] Lacking Zero Address Check for constructor `PoolFactory`

```
constructor(address wethToken) {
+.      require(wethToken != address(0), "Deployment Reverted, WETH Token Must not be Zero Address!");
        i_wethToken = wethToken;
    }
```

[I-3] `PoolFactory::createPool` should use `.symbol()` instead of `.name()`

```
-     string memory liquidityTokenSymbol = string.concat("ts", IERC20(tokenAddress).name());
+     string memory liquidityTokenSymbol = string.concat("ts", IERC20(tokenAddress).symbol());
```